



Smart Resume Screening and Candidate Ranking Engine using Trie, Max Heap, and Hash Maps

CSA0307 – DATA STRUCTURES

Presented By:

ARTHI A- 192525152

SANCIA SANKY – 192572005

Guided by:

DR. K. KIRUTHIKA

OBJECTIVE

- To develop an intelligent resume screening and candidate ranking system.
- To automatically extract candidate information from uploaded resumes.
- To perform efficient skill matching using a Trie data structure. To store and manage candidate records using a Hash Map.
- To calculate ATS scores based on skills, education, experience, projects, and certifications.
- To rank and shortlist candidates using a Max Heap for efficient recruitment.



INTRODUCTION



- Recruitment is a time-consuming process due to the large number of resumes received.
- Manual resume screening is slow and prone to human error.
- Recruiters need an efficient way to identify qualified candidates.
- Data structures can improve search, storage, and ranking efficiency.
- This project automates resume screening and candidate ranking using advanced data structures.

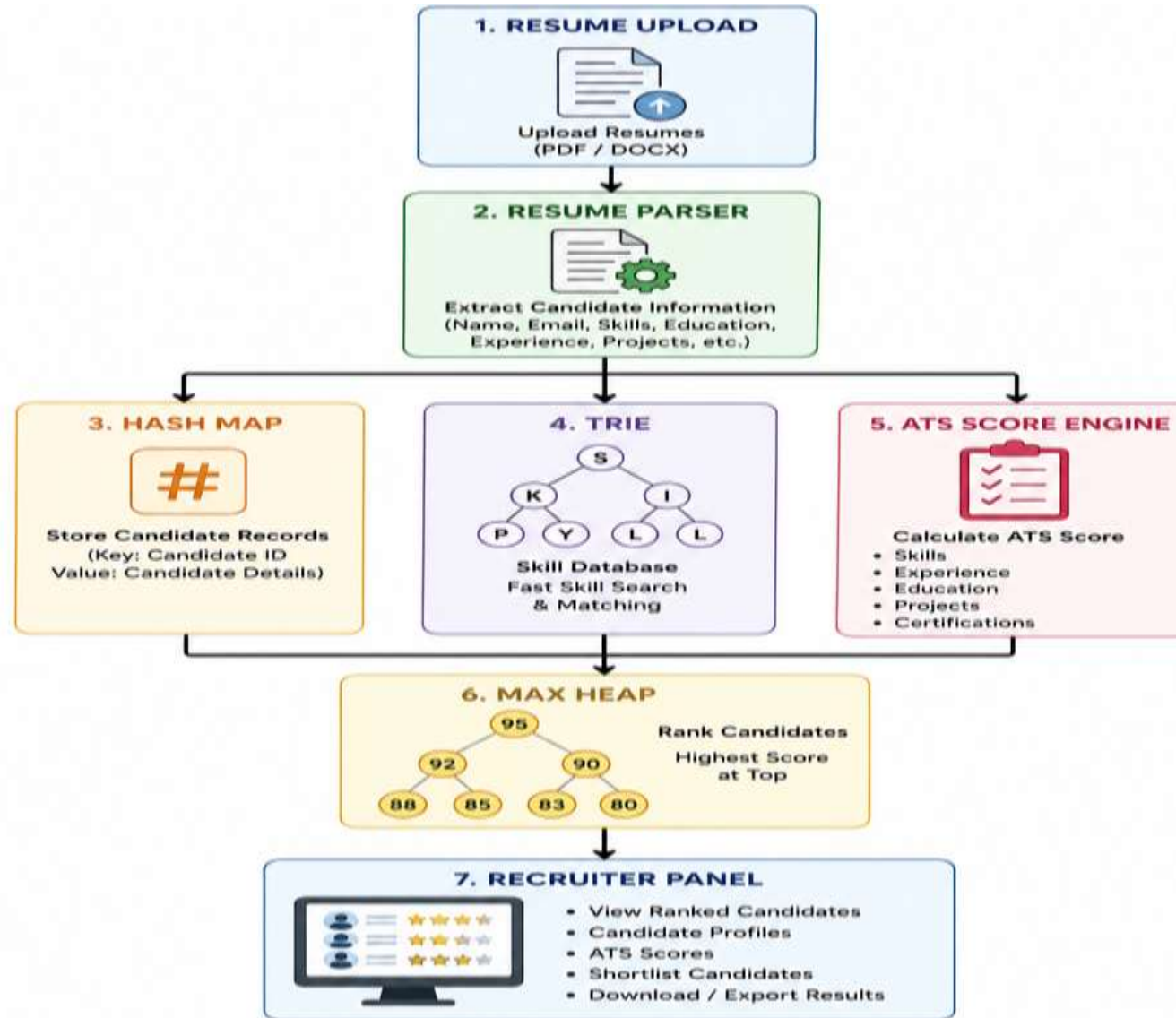


PROBLEM STATEMENT



- Organizations receive a large number of resumes for every job opening.
- Manual resume screening is time-consuming and prone to human error.
- Identifying candidates with the required skills is challenging and inefficient.
- Existing recruitment systems provide limited support for fast skill matching and candidate ranking.
- Recruiters require an intelligent system that can automate resume screening, prioritize candidates, and improve hiring efficiency.

SYSTEM ARCHITECTURE





MODULE 1: RESUME SCREENING AND SKILL MATCHING



Objective

- Extract candidate information.
- Search and match skills efficiently.

Time Complexity

- Trie Search – $O(m)$
- Hash Map Lookup – $O(1)$

Output

- Candidate Profile
- Matching Skills
- Skill Match Percentage

Description

- Upload resumes in PDF/DOCX format.
- Extract candidate details and technical skills.
- Store candidate records in a Hash Map.
- Insert extracted skills into a Trie.
- Compare candidate skills with job requirements.

DATA STRUCTURES USED

Trie

- Stores all technical skills in a tree structure.
- Each node represents a character of a skill.
- Enables fast skill search and prefix matching.
- Reduces search time compared to linear searching.

Hash Map

- Stores candidate details using **Candidate ID** as the key.
- Provides instant access to candidate information.
- Supports fast insertion, retrieval, and updates.



MODULE 2: CANDIDATE RANKING AND SHORTLISTING



Objective

- Rank candidates based on ATS scores.
- Generate a shortlist of the best candidates.

Time Complexity

- Heap Insert – $O(\log n)$
- Get Top Candidate – $O(1)$

Output

- ATS Score
- Candidate Ranking

Description

- Calculate ATS score using skills, experience, education, projects, and certifications.
- Insert candidate scores into the Max Heap.
- Display candidates in descending order of their scores.
- Generate the final shortlist for recruiters.

DATA STRUCTURE USED

Max Heap

- Stores candidates according to their ATS score.
- The candidate with the highest score is always at the root.
- Efficiently maintains ranking when new candidates are added.
- Enables quick retrieval of the top-ranked candidate.



Engineer to Excel

DEVELOPMENT PROCESS



Step 1 – Design User Interface

- Create a recruiter dashboard using Streamlit.

Step 2 – Resume Upload

- Upload PDF/DOCX resumes.

Step 3 – Resume Parsing

- Extract candidate details and technical skills.

Step 4 – Candidate Data Storage

- Store candidate records using a **Hash Map**.

Step 5 – Skill Matching

- Search and match skills using a **Trie**.

Step 6 – ATS Score Calculation

- Calculate scores based on skills, education, experience, projects, and certifications.

Step 7 – Candidate Ranking

- Rank candidates using a **Max Heap**.

Step 8 – Display Results

- Show candidate ranking, ATS scores, and shortlisted candidates.

ADVANTAGES AND APPLICATIONS

Advantages

- Fast resume screening
- Efficient skill matching
- Automatic candidate ranking
- Reduced recruiter workload
- Quick candidate retrieval
- Scalable for large recruitment drives
- Improved hiring accuracy

Applications

- IT Companies
- HR Departments
- Recruitment Agencies
- Campus Placement Cells
- Online Job Portals
- Corporate Recruitment
- Internship Hiring
- Government Recruitment

CONCLUSION

Conclusion

- Automates the resume screening process.
- Improves recruitment efficiency.
- Uses efficient data structures for faster search and ranking.
- Helps recruiters identify the best candidates quickly.

Future Enhancements

- AI-based resume analysis
- Semantic skill matching
- Interview recommendation system
- Resume improvement suggestions
- Integration with LinkedIn and job portals

REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., *Introduction to Algorithms*, 4th Edition, MIT Press, 2022.
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H., *Data Structures and Algorithms in Python*, Wiley, 2013.
3. Deshmukh, A., & Raut, A. (2024). *Applying BERT-Based NLP for Automated Resume Screening and Candidate Ranking*.
4. Gan, C., Zhang, Q., & Mori, T. (2024). *Application of LLM Agents in Recruitment: A Novel Framework for Automated Resume Screening*.
5. Saatci, M., Kaya, R., & Ünlü, R. (2024). *Resume Screening with Natural Language Processing*.



THANK YOU